

Multi-robot Search in 3D Environments using Submodularity with Matroid Intersection Constraints

Yan-Shuo Li¹ and Kuo-Shih Tseng¹

Journal Title
XX(X):1–17
©The Author(s) 2024
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Abstract

The multi-robot search problem is challenging since it involves task allocation, minimal routing, and maximal coverage problems, which are NP-hard. To solve this problem with theoretical guarantees, it is reformulated as a maximal coverage problem subject to the intersection of matroid constraints. The coverage problem is solved by utilizing its submodularity. Additionally, the workload balance is considered to enhance search efficiency. The intersection matroid is composed of a routing constraint and a clustering constraint. The proposed algorithm, Multi-Robot Search with Matroid constraints (MRS_M), achieves $\frac{1}{3}\widehat{OPT}$, where $\widehat{OPT}$ is the optimal performance under spanning-tree structures. Furthermore, Dynamic MRS_M (D-MRS_M) and MRS_M with Hexagonal Packing (MRS_M-Hex) are proposed for unknown and large scale environments, respectively. The experiment results show that the MRS_M approaches outperform state-of-the-art methods in terms of expected time to detection in multi-robot search problems and scale effectively for large search spaces.

Keywords

Submodularity, Matroid, Multi-robot search, Hexagonal packing

Introduction

Multi-robot search is an important problem due to emerging application domains such as target tracking (Schlotfeldt et al. 2018), search and rescue (Jennings et al. 1997), and industrial inspection (Correll and Martinoli 2009). However, the multi-robot search problem is challenging, mainly due to three NP-hard challenges: the maximal coverage problem, multi-robot task allocation, and minimal routing problem. The challenges of these problems are as follows. First, finding the maximal coverage under a budget is computationally intractable (Khuller et al. 1999). Second, the task allocation to multiple robots is another issue (Korsah et al. 2013). Third, each robot in multi-robot search problems needs to solve the traveling salesman problem (TSP) (Zhang and Vorobeychik 2016).

Recent approaches to solving these problems are as follows. For the multi-robot search problem, the researchers propose an efficient path planning algorithm (eMIP) (Singh et al. 2007). The problem is solved by a sequential single robot path planning algorithm. Furthermore, each robot needs to solve the TSP, which results in poor time complexity. A theoretical performance guarantee is provided. However, the search performance deteriorates as the number of robots increases. The problem is reformulated as submodular maximization subject to intersection system constraints (MRSIS) in Li and Tseng (2024). The search problem is solved by generating a set of robot trajectories while considering the balance of task assignments. However, its theoretical performance is not provided. Additionally, solving the routing problem with TSP or Minimum Spanning Tree (MST) at each iteration leads to poor computational time.

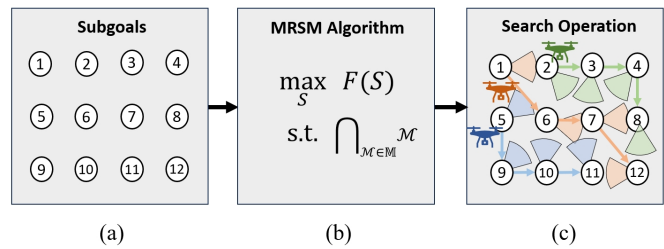


Figure 1. Overview of the proposed multi-robot search system with three robots. (a) An example of subgoals (nodes) in a search environment. (b) The proposed algorithm (MRS_M) finds a solution S that maximizes an objective function F subject to resource constraints $\mathcal{M}$. (c) The circular sectors are the coverage areas and the color arrows (orange, blue, and green) are robot trajectories. The robots follow trajectories generated by MRS_M and find targets in the environment.

For multi-robot task allocation problems, Markov Decision Processes (MDP) over graphs are adopted. In Paul et al. (2022), the researchers propose a reinforcement learning algorithm based on a graph neural network (GNN). The task assignment problem is solved by selecting the node with the highest probability for each robot. However, the routing problem and workload balance are not considered.

For search via learning approaches, the researchers propose a probability density function (PDF) as a reward function that generates a sequence of decisions based on

¹Department of Mathematics, National Central University, Taiwan

Corresponding author:

Kuo-Shih Tseng, National Central University, Taiwan
Email: kuoshih@math.ncu.edu.tw

the reinforcement learning method (Sheng et al. 2022). The search problem and the allocation problem are solved by selecting actions with the maximal individual reward function for each robot. Yet, the graph only considers the unit cost for routing, and the balance of the robots' workloads is not taken into account, potentially resulting in poor task assignment.

To resolve the aforementioned issues, this research proposes a Multi-Robot Search with Matroid constraints (MRSM) algorithm. The key concept of MRSM is to simultaneously solve maximal coverage, task allocation, and minimal routing problems with theoretical guarantees. First, maximal coverage and balanced workloads are considered simultaneously. The coverage function is to cover a larger area for robots, while the balancing function is to measure the equality of the workloads assigned to robots. Second, the clustering constraint is reformulated as a matroid to ensure that the task is divided for K robots. Third, the routing constraint is reformulated as a matroid to boost the theoretical guarantees.

The proposed MRSM method is illustrated in Fig. 1. Given an environment map with subgoals, the goal is to find all targets with multiple robots as soon as possible. In Fig. 1(a), subgoals are evenly distributed in the space. A weighted graph $\mathcal{G}(V, E, w)$ is constructed, where V represents the subgoal set, E represents the edge set, and w is an Euclidean distance function. In Fig. 1(b), MRSM generates a set of trajectories that maximizes the environment coverage and maintains balanced workloads among robots. In Fig. 1(c), the robots visit the subgoals and search for targets in the environment.

The contributions of this research are as follows. First, the submodular maximization under matroid intersection constraints is proposed for multi-robot search problems. Moreover, to solve unknown environment and large scale issues, Dynamic MRSM (D-MRSM) and MRSM with Hexagonal Packing (MRSM-Hex) are proposed, respectively. To the best of our knowledge, this is the first work to propose this objective for these problems. Second, thanks to the submodularity, the theoretical guarantees of MRSIS (Li and Tseng 2024)* and MRSM are proved as $\frac{1}{2+k_G}\overline{OPT}$ and $\frac{1}{3}\overline{OPT}$, respectively, where k_G is the ratio of the maximum to the minimum cardinality ($k_G > 1$), $\overline{OPT}$ is an approximately optimal performance under a graph structure, and $\widetilde{OPT}$ is an approximately optimal performance under spanning-tree structures.[†] Third, simulated and real-world experiments demonstrate that MRSM outperforms the benchmark algorithms (e.g., MRSIS, CapAM (Paul et al. 2022), and PD-FAC (Sheng et al. 2022)) regarding expected time to detection (ETTD) in the multi-robot search problem (e.g., static objects, moving objects, unknown environments). Furthermore, a comprehensive analysis is presented for deployment in specific applications to clarify the parameter influence and scalability.

The primary distinction between MRSIS and MRSM is that, instead of using TSP or MST to solve the routing problem in MRSIS, MRSM expands a spanning tree, resulting in significantly improved computational time. Furthermore, MRSIS models the clustering and routing constraints as independence systems while MRSM models

two constraints as matroids. To the best of our knowledge, the constant factor of the MRSM theoretical bound is better than that of MRSIS. Notice that the theoretical bounds of MRSIS and eMIP (Singh et al. 2007) depend on variables. The key novelty of MRSM, compared to recent approaches, is that MRSM considers clustering and routing budgets within the matroid constraint and achieves a constant theoretical bound.

In this research, some assumptions are made. First, MRSM relies on known environments. A set of subgoals is evenly distributed in the search environments. Second, the coverage at every subgoal is pre-computed since the search method is based on known maps. Third, the perception of robots includes uncertainty, and targets may be occluded in the environment. Although MRSM can be applied in unknown environments, the theoretical guarantees may fail to hold.

This paper is organized as follows: The Related Work section reviews target search methods, multi-robot task allocation, and routing constraints. The Background section provides the foundational knowledge for this research. Problem formulation outlines the problem statement. The Algorithms section details the proposed search algorithms. The Experiments section presents the experimental setup and analyzes the results. The Conclusions section summarizes the findings and discusses future research directions.

Related work

In this section, recent work on target search methods, multi-robot task allocation, and routing constraints is reviewed.

Target Search

Searching for targets in an environment is a sequential decision-making problem. Therefore, search methods based on the type of decision-making algorithm, such as submodular approaches, the frontier-based search, the next-best-view search, the probabilistic search, and Partially Observable Markov Decision Processes (POMDP), can be adopted to generate search strategies.

The submodular method formulates the target search problem as submodular maximization under a resource constraint. Singh et al. (2007) introduce an efficient path planning algorithm (eMIP), which coordinates multiple robots to obtain highly informative paths subject to the robot's path cost. The eMIP finds a solution that achieves at least $\frac{1-1/e}{1+\log_2 N}$, where N is the number of robots. However, the performance deteriorates when more robots are involved. Corah and Michael (2019) formulate the multi-robot exploration problem as submodular maximization under a matroid constraint and propose a distributed sequential greedy assignment (DSGA) algorithm. The partition matroid ensures that each region contains at most one robot. The DSGA algorithm finds a solution achieving at least $\frac{n_r}{n_d}$,

*MRSIS appeared in the proceedings of the 2024 International Conference on Robotics and Automation (ICRA'24), Yokohama, Japan. This manuscript is an extension version of Li and Tseng (2024).

[†]A spanning tree is a tree structure that can be derived from an undirected graph and includes all of the graph's vertices. A minimum spanning tree is a spanning tree whose sum of edge weights is as small as possible.

where n_r and n_d are the number of robots and the number of planning rounds, respectively. Still, the performance is degraded as more robots are involved.

Li and Tseng (2024) propose a Multi-Robot Search with an Intersection System (MRSIS) algorithm. The objective is to generate a set of robot trajectories that maximize the coverage function and balance workloads across robots under clustering and routing cost constraints. The clustering and routing constraints are formulated as an intersection system.

The frontier-based search method utilizes the frontier between the explored and unexplored space to expand existing knowledge. Zhou et al. (2021) propose a hierarchical framework, Fast UAV ExpLoration (FUEL), that supports UAV exploration in complex unknown environments. It improves the exploration efficiency using the incremental frontier information structure. The work is extended to decentralized multi-UAV (Zhou et al. 2023). Instead of allocating frontiers and viewpoints to the UAVs, the task assignment is based on an online hierarchical decomposition to ensure that all UAVs simultaneously explore distinct regions.

The next-best-view (NBV) planning determines the next viewpoint that provides the most valuable information to improve search efficiency. Mittal et al. (2019) adopt an NBV planner for exploration and propose a novel landing site detection algorithm that computes costmaps based on several hazard factors (e.g., terrain flatness, steepness, depth accuracy, and energy consumption information). Lauri et al. (2020) propose a submodular utility function for multi-sensor NBV planning under partition matroid constraints. In addition, the utility function coordinates view selection and prevents overlapping views among multiple sensors.

The probabilistic approach is to estimate the target location under sensor and target motion uncertainty. Robots make decisions based on the probability distribution. Mohamed et al. (2020) present the person search-orienting problem (PSOP). It adopts the user activity probability density functions (APDFs) to generate a search plan to maximize the expected target detection with a limited search time. The approach is then extended to multiple robots by generating a team plan to cooperate effectively (Mohamed et al. 2022). Sheng et al. (2022) propose a probability density factorized (PD-FAC) multi-agent distributional reinforcement learning method that decomposes the PDF of the multi-robot system into a set of individual value distributions. It is guaranteed that the objective function of the overall system's value distribution can be linearly approximated by the same reliability metric defined over the agent's individual value distribution.

Formulating robotic search problems as Partially Observable Markov Decision Processes (POMDP) has recently become a popular method. POMDP considers the search problem where the states of target locations and robot sensors are uncertain. A sequence of actions is generated by maximizing a reward function. Zhu et al. (2020) propose a Dec-POMDP method to find a target in an environment with obstacles. The approach provides a scalable framework for a large number of UAVs. It enables the UAV swarm to cooperate efficiently by sharing limited observations in the mission. Many methods constrain the search space in 2D.

Zheng et al. (2021) present a multi-object search method, 3D Multi-Object Search (3D-MOS), in 3D environments with a frustum-shaped field of view (FOV), which can be applied to mobile robots or drones.

Multi-Robot Task Allocation (MRTA)

The goal of MRTA is to optimize an objective function within a given budget for multiple robots. However, finding an optimal solution is NP-hard (Korsah et al. 2013).

Several research studies have employed submodular maximization with matroid constraints to solve the MRTA problem. This problem involves various challenges, such as the orienteering problem (Gunawan et al. 2016), the intermittent deployment problem (Liu and Williams 2018), and the capacitated vehicle routing problem (Ralphs et al. 2003). If the objective function is submodular, greedy algorithms can find solutions with theoretical guarantees (Nemhauser et al. 1978).

In team surviving orienteers (TSO) problem (Jorgensen et al. 2017a), an independent set of a matroid is selected for maximizing the expected visited nodes by at least one robot. These nodes also ensure that each vehicle reaches its destination with probabilities above a specified threshold. In the environmental monitoring application (Liu and Williams 2019), the multi-robot task allocation problem and the multi-robot intermittent deployment problem are formulated as submodular maximization with matroid intersection constraints. In the surveillance task allocation in urban environments (Williams et al. 2017), a decentralized algorithm, which applies auction methods to assign tasks with matroid constraints, is proposed.

In Linear Sum Assignment Problem (LSAP) (Martello and Toth 1987), the objective is to minimize a cost metric, such as the routing cost or task completion time, while satisfying a one-to-one matching constraint. However, traditional approaches take much time to find the optimal solution. To reduce the time for solving LSAP, Lee et al. (2018) solve the problem by using a feed-forward neural network (FNN) and a convolutional neural network (CNN). The LSAP is decomposed into several classification sub-problems, and FNNs and CNNs are trained to solve each sub-assignment problem. Aironi et al. (2022) utilize a Message Passing Graph Neural Network to solve LSAP. The experiment results demonstrate that the use of GNN outperforms FNN and CNN approaches. Goarin and Loianno (2024) propose a Decentralized Graph Neural Network for Goal Assignment (DGNN-GA). A heterogeneous graph representation is utilized to model the robot communication and the assignment relations between goals and robots. The method achieves good zero-shot generalizability for large teams even in highly restricted communication scenarios.

Recent research on multi-robot task allocation has been paying attention to deep reinforcement learning approaches with GNN. Paul et al. (2022) propose a Capsule Attention-based Mechanism (CapAM), a graph reinforcement learning architecture that encodes graph information such as robot location, task deadline, and elapsed mission time. The approach can be easily scaled to a large number of tasks. Tolstaya et al. (2021) develop a GNN architecture with behavior cloning to solve the coverage problem. Using a sparse representation of the local graph connectivity, the

approach can scale to larger maps and teams. Zhang et al. (2022a) propose a Hierarchical-Hops Graph Neural Network (H2GNN), which evaluates the importance of graph nodes at different hops through the multi-head attention mechanism. To improve exploration efficiency, multi-agent reinforcement learning is applied to learn collaborative strategies.

Routing Constraints

Submodular optimization has been explored with extensive applications in various domains, such as sensor placement (Krause and Guestrin 2007), viral marketing (Golovin and Krause 2011), and robotic search problems (Tsai et al. 2019). To apply submodular optimization in realistic environments, different constraints (e.g., cardinality (Nemhauser et al. 1978), additive budget (Khuller et al. 1999), and routing (Zhang and Vorobeychik 2016)) must be considered.

Zhang and Vorobeychik (2016) propose a generalized cost-benefit (GCB) greedy algorithm to solve two NP-hard problems: monotone submodular maximization and routing path minimization (e.g., TSP). GCB finds a solution that achieves $\frac{1}{2}(1 - \frac{1}{e})\widehat{OPT}$ optimum, where $\widehat{OPT}$ is the approximated optimum. However, there is a gap between $\widehat{OPT}$ and the optimal solution (OPT). To improve the theoretical guarantee of GCB, Lin and Tseng (2023) propose Tree-Structured Fourier Supports Set (TS-FSS) algorithms that combine the characteristics of submodularity and sparsity of routing trees to boost the theoretical bound.

Besides, Zhang et al. (2022b) investigate the non-monotone submodular maximization with the k -independence system routing constraint, where k is loosely upper-bounded by the size of the ground set. An iterated two-stage algorithm is presented to obtain a $[\frac{1}{4k}, 1 + \theta]$ -bicriterion approximation solution, where θ is an error parameter within $(0, 1)$.

Recent approaches solve the routing problem by learning heuristics. Kool et al. (2019) propose an attention model and train the model using REINFORCE with a greedy rollout baseline. The attention model generates a set of trajectories based on the node coordinates of a graph. Utilizing learning-based approaches offers the advantage of scalability to larger problem instances. Kaempfer and Wolf (2018) propose a model based on PointNet (Qi et al. 2017). The model encodes the locations of depots and salesmen. The outcome can be interpreted as a solution to the linear relaxation of the problem. Besides, they employ a beam search algorithm to obtain a feasible integer solution.

Background

To formulate the multi-robot search problem, some preliminaries are introduced. Submodularity (Def. 1) and independence system (Def. 2) are adopted to formulate the objective function and the robot constraints, respectively. The objective function consists of the coverage and balancing function (Thm. 6). The constraint includes clustering of the search environment (Thm. 4). The advantage of formulating the submodular maximization subject to the intersection of an independence system (Thm. 1) is that the theoretical guarantee is given (Thm. 3). To further improve the performance, Def. 4 introduces the

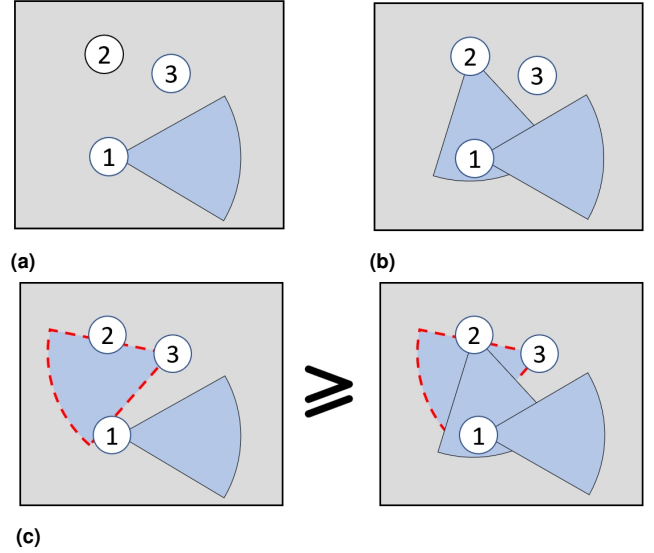


Figure 2. Illustration of submodularity under coverage scenario. The geographical locations of sensors are denoted as decimal numbers, while the blue and gray areas correspond to the covered and uncovered regions, respectively. (a) $f(S_A)$ is the covered area of S_A , where $S_A = \{1\}$. (b) $f(S_B)$ is the covered area of S_B , where $S_B = \{1, 2\}$. (c) The marginal gain of the coverage function f is marked as red dashed lines when $s = \{3\}$ is added to the current set $S_A = \{1\}$ and $S_B = \{1, 2\}$. Mathematically, $f(S_A \cup s) - f(S_A) \geq f(S_B \cup s) - f(S_B)$.

concept of a matroid.

Definition 1. (Submodularity) (Nemhauser et al. 1978) A function $f: 2^N \rightarrow \mathbb{R}^+$ is submodular if and only if $\forall S \subseteq T \subseteq N, \forall e \in N \setminus T, f(S \cup \{e\}) - f(S) \geq f(T \cup \{e\}) - f(T)$.

Various functions satisfy submodularity, including the coverage function (Krause and Guestrin 2007), the mutual information function (Singh et al. 2007), and the tree-structure routing function (Lin and Tseng 2023). In the coverage function case (see Fig. 2), let $S = \{1, 2, 3\}$ be the ground set and $S_A = \{1\}$, $S_B = \{1, 2\}$ be two different sets selected from the ground set, i.e., $S_A \subseteq S_B \subseteq S$. In Fig. 2 (a)(b), $f(S_A)$ and $f(S_B)$ depict the regions observed by the sensor at locations of $\{1\}$ and $\{1, 2\}$. When a new set $s = \{3\}$ is added to both S_A and S_B , the marginal gain of the coverage function f is the additional regions covered, which is the red dashed line area shown in Fig. 2(c). The diminishing return property suggests that incorporating a set into the smaller set results in a greater marginal gain.

Definition 2. (Independence System) (Korte and Hausmann 1978) An independence system is a pair $(E, \mathcal{I})$, where E is a finite set called the ground set and $\mathcal{I}$ is a family of subsets of E called independent sets such that:

1. (non-emptiness) $\emptyset \in \mathcal{I}$;
2. (downward closure) if $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$.

An illustrative example is as follows. Consider a ground set $E = \{a, b, c, d, e, f, g, h\}$ and an independent set $\mathcal{I} =$

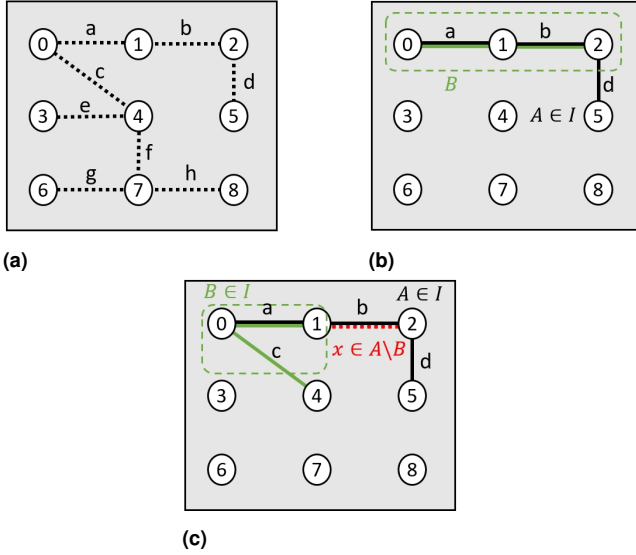


Figure 3. Illustration of the downward closure and the exchange property. The vertices represent the geographic locations of sensors, and the edges are the distance between two vertices. (a) The ground set $E = \{a, b, c, d, e, f, g, h\}$ (the black dashed lines) (b) $A = \{a, b, d\}$ (the black solid lines) is a set that satisfies the independent set $\mathcal{I}$. $B = \{a, b\}$ (the green solid lines) is a subset of A . (c) Two sets $A = \{a, b, d\}$ (the black solid lines) and $B = \{a, c\}$ (the green solid lines) satisfy the independent set $\mathcal{I}$, where $|A| > |B|$. Then there exists an element $x \in A \setminus B$ such that adding to the smaller set B still satisfies the condition of the independent set $\mathcal{I}$, e.g., $x = b$ (the red dashed lines).

$\{\{a, b, c\}, \{a, b, d\}, \{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{a\}, \{b\}, \{c\}, \{d\}, \emptyset\}$, shown in Fig. 3(a). Let $A \in \mathcal{I}$ be a set, and $B \subseteq A$ be a subset. In Fig. 3(b), the downward closure property states that if $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$.

Definition 3. (*k*-independence System) (Korte and Hausmann 1978) Given a ground set E , an independence family $\mathcal{I}$ and a set $Y \subseteq E$. Let $r(Y) = \{A \in \mathcal{I} \mid A \subseteq Y \text{ and there is no } A' \in \mathcal{I} \text{ such that } A \subset A' \subseteq Y\}$. Then $(E, \mathcal{I})$ is a *k*-independence system if for all $Y \subseteq E$,

$$\frac{\max_{A \in r(Y)} |A|}{\min_{A \in r(Y)} |A|} \leq k,$$

where $k \geq 1$ and $|\cdot|$ is defined as the number of elements. As a special case, if $k = 1$, then $(E, \mathcal{I})$ is a matroid.

To clarify the concept of *k*-independence system, consider a ground set $E = \{1, 2, 3, 4, 5\}$ and an independence family $\mathcal{I} = \{\{1, 2, 3\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 4\}, \{1\}, \{2\}, \{3\}, \{4\}, \emptyset\}$. Let $Y = \{2, 3, 4\}$ be a subset of E . By definition, $r(Y) = \{\{3\}, \{2, 4\}\}$. The ratio of the maximal cardinality to the minimal cardinality is 2. Thus, $k \geq 2$.

Definition 4. (Matroid) (Nemhauser et al. 1978) A matroid $\mathcal{M}$ is a pair $(E, \mathcal{I})$, where E is a finite set called the ground set and $\mathcal{I}$ is a family of subsets of E called independent sets, with the following properties:

1. (downward closure) if $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$;

2. (exchange property) if $A \in \mathcal{I}, B \in \mathcal{I}$ and $|A| > |B|$, then there exists $x \in A \setminus B$ such that $B \cup \{x\} \in \mathcal{I}$.

An example of a matroid is illustrated in Fig. 3. Consider $\mathcal{M} = (E, \mathcal{I})$, where $E = \{a, b, c, d, e, f, g, h\}$ is a ground set and $\mathcal{I} = \{\{a, b, c\}, \{a, b, d\}, \{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{a\}, \{b\}, \{c\}, \{d\}, \emptyset\}$ is an independent set, as shown in Fig. 3(a). If $\mathcal{M}$ is a matroid, the independent set $\mathcal{I}$ must satisfy the downward closure and the exchange property. Let $A \in \mathcal{I}$ be a set and $B \subseteq A$ be a subset, shown in Fig. 3(b). The downward closure property states that if $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$. In Fig. 3(c), consider two sets $A \in \mathcal{I}$ and $B \in \mathcal{I}$, where $|A| > |B|$. By the exchange property, there exists an element $x \in A \setminus B$ such that $B \cup \{x\} \in \mathcal{I}$.

Various types of matroids can be applied in robotics, including the uniform matroid (Jorgensen et al. 2017b), the graphic matroid (Kruskal 1956), and the partition matroid (Liu and Williams 2019).

A uniform matroid, defined as $\mathcal{M}_u(E, \mathcal{I}_u)$, where $\mathcal{I}_u = \{X \subseteq E : |X| \leq K\}$, can be used to formulate the TSO problem (Jorgensen et al. 2017b). Given a graph where each edge is weighted by the probability of survival during traversal, the goal is to find a set of paths for K robots that maximize the expected number of nodes visited, while ensuring that each robot's probability of surviving to its destination meets specified constraints.

A graphic matroid, defined as $\mathcal{M}_g(E, \mathcal{I}_g)$, where $\mathcal{I}_g = \{X \subseteq E : X \text{ does not contain cycle}\}$, can be used to solve the MST problem (Kruskal 1956). The goal is to iteratively select edges with minimal weight, while the independent set ensures that the selected edges do not form cycles.

A partition matroid, defined as $\mathcal{M}_p(E, \mathcal{I}_p)$, where $E = E_1 \cup \dots \cup E_n$ and $\mathcal{I}_p = \{X \subseteq E : |X \cap E_k| \leq l_k\}$, is widely adopted to solve multi-robot coordination problems (Liu and Williams 2019). In multi-robot task allocation, tasks are divided into disjoint sets, and the independent set ensures that no more than l_k robots are deployed at time k .

Theorem 1. (Intersection of independence systems) (Mestre 2015) The intersection of a k_1 -independence system and a k_2 -independence system is a $(k_1 + k_2)$ -independence system.

To illustrate the concept, let $(E, \mathcal{I}_1)$ and $(E, \mathcal{I}_2)$ be a k_1 -independence system and a k_2 -independence system, respectively. The goal is to find the intersection of $(E, \mathcal{I}_1)$ and $(E, \mathcal{I}_2)$, namely $(E, \mathcal{I}_1 \cap \mathcal{I}_2)$. Then $(E, \mathcal{I}_1 \cap \mathcal{I}_2)$ is a $(k_1 + k_2)$ -independence system.

Theorem 2. (Intersection of matroids) (Korte and Hausmann 1978) Consider the matroid intersection system

$$\bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M} \triangleq \left(\mathcal{V}, \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M} \right), \quad (1)$$

where $\mathbb{M}$ is a set of matroidal independence systems, $\mathcal{M}$ is a matroid, and $\mathcal{V}$ is a ground set. Eq. (1) models any arbitrary

independence system.

Theorem 3. (Theoretical bound of a matroid intersection) (Fisher et al. 1978) Given a submodular monotone set function F and a set of matroidal independence systems $\mathbb{M}$, maximizing F under an intersection of matroids with the greedy algorithm yields a solution X . The performance of the set X is

$$F(X) \geq \frac{1}{|\mathbb{M}| + 1} F(X^{opt}),$$

where $|\mathbb{M}|$ is the cardinality of $\mathbb{M}$ and X^{opt} is an optimal solution.

Consider a set of matroidal independence systems $\mathbb{M} = \{\mathcal{M}_1, \mathcal{M}_2\}$. The cardinality of $\mathbb{M}$ is 2. Therefore, greedy algorithms find a solution X such that the performance of X achieves $\frac{1}{3} F(X^{opt})$.

In multi-robot search scenarios, the search environment must be partitioned into smaller segments to accommodate multiple robots. Ensuring an equitable distribution of workloads among robots is a key consideration for efficient task assignments. Therefore, the coverage and balancing function (Li and Tseng 2024) and the clustering constraint (Liu et al. 2013) are introduced.

Theorem 4. (Clustering constraint) (Liu et al. 2013) Let E be the edge set and $S \subseteq E$ be the set of subsets. $\mathcal{M}_C = (E, \mathcal{I}_C)$ is matroidal, where $\mathcal{I}_C = \{S \subseteq E : N \geq n\}$, N is the number of clusters, and n is the clustering budget.

Theorem 5. (Submodularity of the balancing function) (Liu et al. 2013) Given a graph $\mathcal{G} = (V, E)$ and a partition set $S = \{S_1, S_2, \dots, S_N\}$, the balancing function $\mathcal{B} : 2^E \rightarrow \mathbb{R}$ is a monotonically increasing submodular function and is defined as follows:

$$\mathcal{B}(S) = - \sum_i p_S(i) \log(p_S(i)) - N,$$

where $p_S(i) = \frac{|S_i|}{|V|}$, $i = \{1, \dots, N\}$ and N is the number of connected components in the graph.

The balancing function encourages the clusters to have similar sizes. Fig. 4 shows the balancing function of two topologies. The balancing function shown in Fig. 4(a) has a higher value compared to that in Fig. 4(b). In other words, the higher balancing value has more balanced clustering.

Theorem 6. (Submodularity of a coverage and balancing function) (Li and Tseng 2024) Let S be a set, $\lambda \in [0, 1]$ be a constant, $f : 2^S \rightarrow \mathbb{R}^+$ be a coverage function, and $\mathcal{B} : 2^S \rightarrow \mathbb{R}^+$ be a balancing function. The coverage and balancing function $F(S) = f(S) + \lambda \mathcal{B}(S)$ is submodular.

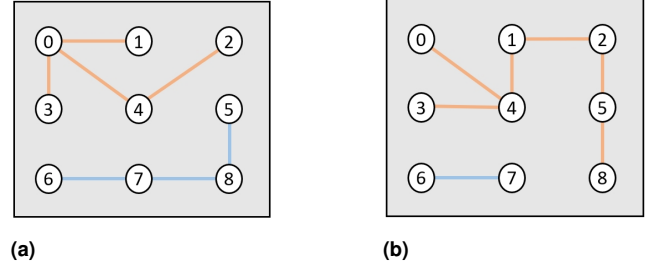


Figure 4. Illustration of the balancing function. Two clusters are denoted by orange and blue colors.

(a) $\mathcal{B}(S_A) = -1.01$ is the balancing value of S_A , where $S_A = \{S_1, S_2\}$, $S_1 = \{0, 1, 2, 3, 4\}$, and $S_2 = \{5, 6, 7, 8\}$.
(b) $\mathcal{B}(S_B) = -1.24$ is the balancing value of S_B , where $S_B = \{S_1, S_2\}$, $S_1 = \{0, 1, 2, 3, 4, 5, 8\}$, and $S_2 = \{6, 7\}$.

Problem Formulation

Given a weighted graph $\mathcal{G} = (V, E, w)$ and an objective function $F : 2^E \rightarrow \mathbb{R}^+$, where V is a set of vertex, E is a set of edges and $w : E \rightarrow \mathbb{R}^+$ is a distance function, the goal is to find a subset $S \subseteq E$ that maximizes environmental coverage and balances robot workloads subject to a constraint. The objective function $F(S) = f(S) + \lambda \mathcal{B}(S)$ is defined as a linear combination of a coverage function f and a balancing function $\mathcal{B}$, where $\lambda \in [0, 1]$ is the weight of the balancing function[‡]. In 3D environments, the coverage function is defined by calculating the ratio of the number of covered voxels to the total number of environmental voxels.

Two variant formulations with known world maps are considered: Multi-Robot Search with Independence System (MRSIS) and Multi-Robot Search with Matroid (MRSM). The following two subsections prove the theoretical guarantees of MRSIS and MRSM.

Multi-Robot Search with Independence System (MRSIS)

Problem 1. (MRSIS) (Li and Tseng 2024) Given an objective function F , a set of independence system $\mathbb{I} = \{I_G, I_C\}$, the number of robots $n \in \mathbb{N}^+$, and a set of routing constraints for each robot $l = \{l_i \in \mathbb{R}^+\}_{i=1}^n$, find a subset $S \subseteq E$ that maximizes F under an intersection of independence systems

$$\begin{aligned} \max_S \quad & F(S) \\ \text{s.t.} \quad & S \in \bigcap_{I \in \mathbb{I}} I, \end{aligned} \quad (2)$$

The intersection system $\bigcap_{I \in \mathbb{I}} I$ is defined as an intersection of all elements in the set of independence systems. $\mathbb{I} = \{I_G, I_C\}$ is defined as a set of independence systems, where $I_G = (E, \mathcal{I}_G)$ models the robots' routing constraint and $I_C = (E, \mathcal{I}_C)$ models the cluster constraint.

The following theorems and corollaries prove that the routing and clustering constraints are k -independence systems. Then, the theoretical performance guarantee of MRSIS is provided.

[‡]Both f and $\mathcal{B}$ functions are normalized.

Theorem 7. (Routing constraint of a k -independence system) The routing constraint for multiple robots $I_G = (E, \mathcal{I}_G)$ is a k_G -independence system, where k_G is the ratio of the maximum to the minimum cardinality ($k_G > 1$). The independent set is defined as $\mathcal{I}_G = \{S \subseteq E : c(S \cap X_i) \leq l_i, \forall i \in \{1, \dots, n\}\}$, where $c: 2^E \rightarrow \mathbb{R}^+$ is a routing cost function (e.g., TSP or MST solver), $X_i \subseteq S$ is the set of the i^{th} cluster and l_i is a routing budget.

Proof. To prove that I_G is an independence system, two properties described in Def. 2 must hold.

(Non-emptiness property) Consider a set $B_1 = \emptyset$. Let $A = \{1, \dots, n\}$ and $m_i = c(B_1 \cap X_i), \forall i \in A$ be the routing costs of the i -th cluster and X_i be a set of cluster i . Since B_1 is an empty set, it implies that $m_i = 0, \forall i \in A$. Thus, B_1 belongs to the independent set $\mathcal{I}_G$.

(Downward closure property) Consider a set $B_1 \subseteq E$ and $B_1 \in \mathcal{I}_G$. Let $m_i = c(B_1 \cap X_i), \forall i \in A$ be the routing costs of the i -th cluster and X_i be a set of cluster i . Since $B_1 \in \mathcal{I}_G$, this implies that $c(B_1 \cap X_i) \leq l_i, \forall i \in A$. Therefore, $m_i \leq l_i, \forall i \in A$. Let $B_2 \subseteq B_1$ and $n_i = c(B_2 \cap X_i), \forall i \in A$. Since B_2 contains edges no more than B_1 , the routing costs $n_i \leq m_i$. As a result, $n_i \leq m_i \leq l_i$. Therefore, B_2 belongs to the independent set $\mathcal{I}_G$.

Since the independent set $\mathcal{I}_G$ satisfies the non-emptiness and downward closure properties, $I_G = (E, \mathcal{I}_G)$ is a k_G -independence system. If I_G does not satisfy the exchange property (Def. 3), k_G will be greater than 1.

To prove that $k_G > 1$, consider two sets $B_1 \in \mathcal{I}$ and $B_2 \in \mathcal{I}$, where $|B_1| > |B_2|$. Let $s = B_1 \setminus B_2$, $m_i = c(B_1 \cap X_i) \leq l_i, \forall i \in A$ and $n_i = c(B_2 \cap X_i) = l_i, \forall i \in A$. If $X \neq \emptyset$, adding any element $x \in X$ to B_2 forms a new set B'_2 . It is obvious that $B'_2 \notin \mathcal{I}$ since $n_i = l_i \leq c(B'_2 \cap X_i)$. Therefore, I_G does not satisfy the exchange property. k_G is greater than 1.

To illustrate the k_G , an example is as follows: Note that k_G depends on the structure of a graph and the routing budget l_i . For simplicity, consider a single-robot case, where $n = 1$. Fig. 5 illustrates the concept of k_G . Let the ground set E include all edges shown in Fig. 5(a) and assume $l_i = 12$. The set $\mathcal{I}_G$ consists of subsets where the routing length is less than 12. Recall the definition of k -independence system (Def. 3). Let Y be a subset of E . It holds that $\max_{A \in r(Y)} |A| = 4$ because the maximal cardinality satisfying the routing budget is 4, as shown in Fig. 5(b). Additionally, $\min_{A \in r(Y)} |A| = 3$, since the minimal cardinality that meets $r(Y)$ within the routing budget is 3, as shown in Fig. 5(b). Therefore, $k_G = \frac{4}{3}$.

Corollary 1. (Clustering constraint as a 1-independence system) Let E be an edge set and $S \subseteq E$ be a subset. $I_C = (E, \mathcal{I}_C)$ is an 1-independence system, where $\mathcal{I}_C = \{S \subseteq E : N \geq n\}$, N is the number of clusters, and n is the number of robots.

Proof. Since I_C is a matroid (Thm. 4) and a matroid satisfies the properties of the independence system (Def. 2), the clustering constraint I_C is an 1-independence system.

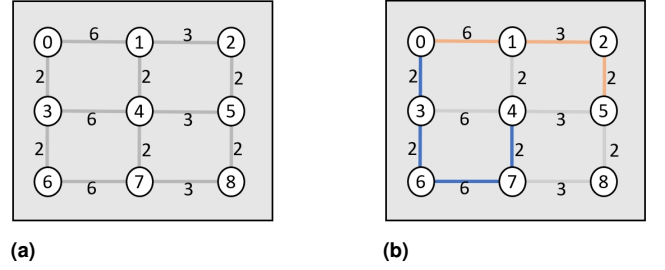


Figure 5. Illustration of k_G -independence system.

(a) The ground set E (gray lines). (b) The blue line provides an example where $\max_{A \in r(Y)} |A| = 4$, while the orange line illustrates an example where $\min_{A \in r(Y)} |A| = 3$.

To explain the concept of the clustering constraint, suppose that there are two robots ($n = 2$). Fig. 4(a) and (b) satisfy the condition of $\mathcal{I}_C$ since the graph involves two clusters.

Theorem 8. (Lower bound of MRSIS) Let F be the coverage and balancing function, $\mathbb{I}$ be a set of k -independence systems, S be the solution of maximizing F subject to the intersection of $\mathbb{I}$ via greedy algorithms. The performance of S is

$$F(S) \geq \frac{1}{2 + k_G} F(\overline{S}^{opt}),$$

where $k_G > 1$ and $\overline{S}^{opt}$ is an approximately optimal solution depending on the TSP or MST solver.

Proof. In Thm. 6, the coverage and balancing function F satisfies the submodularity. Let $\mathbb{I} = \{I_C, I_G\}$ be a set of k -independence systems. In Cor. 1, the clustering constraint I_C is a 1-independence system. In Thm. 7, the general multi-robot routing constraint I_G is a k_G -independence system. By Thm. 1, the performance of MRSIS achieves $\frac{1}{2+k_G} \overline{OPT}$, where $\overline{OPT}$ is an approximately optimal performance that depends on the routing cost function (e.g., TSP or MST).

To improve the performance of MRSIS, the set of independence systems is reformulated into the set of matroids. The novel objective function is called Multi-Robot Search with Matroid (MRSIM).

Multi-Robot Search with Matroid (MRSIM)

Problem 2. (MRSIM) Given an objective function F , a set of matroidal independence systems $\mathbb{M} = \{\mathcal{M}_R, \mathcal{M}_C\}$, the number of robots $n \in \mathbb{N}^+$, and a set of routing constraints for each robot $l = \{l_i \in \mathbb{R}^+\}_{i=1}^n$, find a set $S \subseteq E$ that maximizes F under an intersection of matroidal independence systems

$$\begin{aligned} \max_S \quad & F(S) \\ \text{s.t.} \quad & S \in \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M}, \end{aligned} \quad (3)$$

$\mathbb{M} = \{\mathcal{M}_R, \mathcal{M}_C\}$ is defined as a set of matroidal independence systems, where $\mathcal{M}_R = (E, \mathcal{I}_R)$ and $\mathcal{M}_C = (E, \mathcal{I}_C)$ are the robots' routing constraint under tree structure and the clustering constraint, respectively.

The routing matroid $\mathcal{M}_R$ constrains the trajectory length of robots while constructing spanning trees. The independent set $\mathcal{I}_R$ is a subset of E which satisfies:

1. $T \subset S$, where $T \subseteq E$ and $S \subseteq E$ are acyclic, and $|T| < |S|$;
2. $c_{st}(S \cap X_i) \leq l_i, \forall i \in \{1, 2, \dots, n\}$,

where $X_i \subseteq S$ is the set of the i^{th} cluster and $c_{st} : 2^E \rightarrow \mathbb{R}^+$ is a routing cost function for a spanning tree.

The clustering matroid $\mathcal{M}_C$ is defined the same as in MRSIS. The independent set $\mathcal{I}_C$ is defined as

$$\mathcal{I}_C = \{S \subseteq E : N \geq n\},$$

where N is the number of clusters and n is the number of robots.

In practice, the number of clusters N will be the same as the number of robots n due to the maximization of the balancing function. For example, there are 3 clusters in Fig. 6(a), with a balancing value of -1.5. If the next edge extends from existing clusters, such as by adding the edge (0, 1), the balancing value becomes -1.48. However, if the next edge creates a new cluster, such as by adding the edge (1, 2), the balancing value decreases to -2.08. Therefore, MRSM will extend the next edge from existing clusters.

The following Theorems introduce matroid constraints and the theoretical guarantee of MRSM.

Theorem 9. (Routing constraint under tree structure) Let E be an edge set and T and S be any subset of E . $\mathcal{I}_R$ is the set of subsets, $S \subseteq E$, which satisfies:

1. $T \subset S$, where T and S are acyclic and $|T| < |S|$;
2. $c_{st}(S \cap X_i) \leq l_i, \forall i \in \{1, \dots, n\}$, where $c_{st} : 2^E \rightarrow \mathbb{R}^+$ is a routing cost function for a spanning tree, $X_i \subseteq S$ is the set of the i^{th} cluster, and l_i is a routing budget.

The routing constraint $\mathcal{M}_R = (E, \mathcal{I}_R)$ is a matroid.

Proof. To prove that $\mathcal{M}_R$ is a matroid, two properties described in Def. 4 must hold.

(Downward closure property) Consider a set $B_1 \subseteq E$ and $B_1 \in \mathcal{I}_R$. Let $A = \{1, \dots, n\}$ and $m_i = c_{st}(B_1 \cap X_i), \forall i \in A$ be the routing costs of the i -th cluster, and X_i be the set of the i -th cluster. Since $B_1 \in \mathcal{I}_R$, this implies that B_1 is a tree (acyclic graph) and $m_i \leq l_i, \forall i \in A$. Let $B_2 \subseteq B_1$ and $n_i = c_{st}(B_2 \cap X_i), \forall i \in A$. Since B_2 contains edges no more than B_1 , the routing costs must be $n_i \leq m_i$. In addition, removing any edge from B_1 forms B_2 which does not form a cycle. As a result, $n_i \leq m_i \leq l_i$ and B_2 is acyclic. Therefore, $\mathcal{I}_R$ satisfies the downward closure property.

(Exchange property) Consider two sets $B_1 \in \mathcal{I}_R$ and $B_2 \in \mathcal{I}_R$, where $|B_1| > |B_2|$. Since B_1 and B_2 satisfy the condition of the independent set and $|B_1| > |B_2|$, it implies that $B_2 \subset B_1$. Let $m_i = c_{st}(B_1 \cap X_i)$ and $n_i = c_{st}(B_2 \cap X_i), \forall i \in A$. If $B_2 \subset B_1$, it is clear that $n_i \leq m_i \leq l_i$. Besides, removing any edge from B_1 forms B_2 , which is acyclic. Therefore, $\mathcal{I}_R$ satisfies the exchange property.

Since the independent set $\mathcal{I}_R$ satisfies the downward closure properties and the exchange property, $\mathcal{M}_R = (E, \mathcal{I}_R)$ is a matroid.

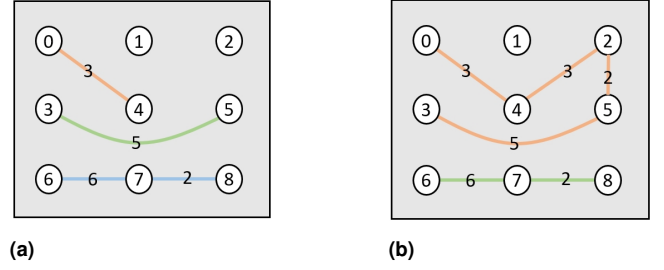


Figure 6. Illustration of the intersection system. The vertices and lines represent the subgoals and the distances between two vertices, respectively. Each color line denotes a cluster. (a) $S_A \in \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M}$ contains three clusters that satisfy the condition of the intersection system. (b) $S_B \notin \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M}$ contains two clusters that violate the condition of the intersection system due to the under-budget of the clustering and the over-budget of routing constraints.

Definition 5. (Matroidal independence systems of routing and clustering constraints) Given a tree-structured routing constraint $\mathcal{M}_R$ with a set of routing budgets $\{l_i\}_{i=1}^n$ and a clustering constraint $\mathcal{M}_C$ with n robots. A set of matroidal independence systems is defined as $\mathbb{M} = \{\mathcal{M}_R, \mathcal{M}_C\}$.

The routing matroid $\mathcal{M}_R$ constrains the trajectory length of robots via constructing spanning trees, while the clustering matroid $\mathcal{M}_C$ divides the ground set into groups for robots.

To illustrate the concept, two cases are illustrated as follows. Fig. 6 illustrates the intersection system of $\mathcal{M}_R = (E, \mathcal{I}_R)$ and $\mathcal{M}_C = (E, \mathcal{I}_C)$. Let E be the set of edges between any vertex, $(a, b) \in E$ be an undirected edge, $\mathbb{M} = \{\mathcal{M}_R, \mathcal{M}_C\}$ be the set of matroidal independence systems, $l_i = 10$ be the routing constraint and $n = 3$ be the minimum number of clusters. In Fig. 6(a), let $S_A = \{S_1, S_2, S_3\}$, where $S_1 = \{(0, 4)\}$, $S_2 = \{(3, 5)\}$, and $S_3 = \{(6, 7), (7, 8)\}$. S_A satisfies the intersection system since S_A is acyclic, $|S_A| \geq n$ and $c(S_A \cap S_i) \leq l_i$, where $i = \{1, 2, 3\}$. In Fig. 6(b), let $S_B = \{S_1, S_2\}$, where $S_1 = \{(0, 4), (4, 2), (2, 5), (3, 5)\}$ and $S_2 = \{(6, 7), (7, 8)\}$. S_B does not satisfy the intersection system since $|S_B| < n$ and $c(S_B \cap S_1) > l_i$.

Theorem 10. (Lower bound of MRSM) Let F be the coverage and balancing function, $\mathbb{M}$ be a set of matroidal independence systems, S be the solution of maximizing F subject to the intersection of $\mathbb{M}$ via greedy algorithms. The performance of S is

$$F(S) \geq \frac{1}{3} F(\widetilde{S}^{opt}),$$

where $\widetilde{S}^{opt}$ is the optimal solution under tree structure.

Proof. The coverage and balancing function F is submodular (Thm. 6) and each element in $\mathbb{M}$ is a matroid (Thm. 4 and Thm. 9). By Thm. 3, the performance of the proposed method is guaranteed to find a solution that achieves $\frac{1}{3} \widetilde{OPT}$, where $\widetilde{OPT}$ is the optimal performance under tree structure.

Algorithms

This research proposes an MRSM algorithm, which is an improved version of MRSIS. The objective is to solve Eq. (3). The algorithm generates k trajectories for k robots.

Multi-robot Search Algorithm

MRSM is presented in Alg. 1. Line 1 is to initialize the selected edge set S . Lines 2-3 define the objective function and the set of matroidal independence systems, respectively. Lines 4-7 initialize the edges with a time complexity of $\mathcal{O}(n)$. Inspired by Kruskal's algorithm (Kruskal 1956), the initial edge e_i is sampled according to the minimum weight of the edge. Lines 9-16 iterate over the edge set to greedily select the optimal edge, with a time complexity of $\mathcal{O}(|E|^2)$. The main difference between MRSIS and MRSM is in lines 11-14. In MRSIS, the graph $S \cup \{e^*\}$ is transformed into robot trajectories by a TSP or MST solver. In MRSM, the maximal marginal gain is selected to ensure that S remains a spanning tree.

The advantage is that MRSM rules out redundant processes (e.g., solving TSP or constructing MST), which improves the computational time. The computational complexity of MRSM is $\mathcal{O}(|E|^2)$. On the other hand, the computational complexity of MRSIS is $\mathcal{O}(|E|^2 + |E||V|^2)$ for both TSP and MST.

The key innovation of the MRSM algorithm is that it expands the spanning trees with the matroid property. Hence, MRSM adopts Kruskal's algorithm since Kruskal's algorithm inherently follows a matroid structure. In constructing an MST, Kruskal's algorithm selects the edge with the minimum weight from the ground set. It verifies that adding an edge to the solution does not form cycles. The primary difference between MRSM and Kruskal's algorithm is their objectives. Kruskal's algorithm seeks to generate a minimum spanning tree, while MRSM discovers a spanning tree that maximizes the objective function under clustering and routing budgets.

To adapt dynamic MRSM (D-MRSM) for online exploration, the exploration in unknown maps is presented in Alg. 2. The main difference between known maps and unknown maps lies in lines 4-13 with a time complexity of $\mathcal{O}(K)$, where K is the maximal number of edges for all robots to exceed the routing budget l_i . Line 4 checks whether all robots' trajectories exceed their routing budgets. Line 5 constructs the edge set $\tilde{E}$ by randomly sampling obstacle-free trajectories to the next subgoal, where $\tilde{E}$ contains the trajectories from the current robots' positions to the next subgoal positions. Lines 6-12 remain the same as Alg. 1, with a time complexity of $\mathcal{O}(|\tilde{E}|^2)$. The computational complexity of D-MRSM under unknown maps is $\mathcal{O}(K|\tilde{E}|^2)$.

Computational Complexity in Large-Scale Scenarios

To alleviate the computational load of MRSM in large-scale environments, the key is to select the graph edges of MRSM (see line 9 in Alg. 1). MRSM with Hexagonal Packing (MRSM-Hex) is introduced. In MRSM, the weighted graph G is a complete graph, while in MRSM-Hex, the graph G is a hexagonal packing graph (see Fig. 7). The hexagonal

packing structure ensures full coverage of the area within the FOV.

Since the graph structures of MRSM and MRSM-Hex are different, their computational complexity and flexibility are impacted directly. MRSM utilizes a complete graph, where each vertex connects to every other vertex, resulting in an edge set of size $\mathcal{O}(|V|^2)$. This leads to a computational complexity of $\mathcal{O}(|V|^4)$, making it impractical for large-scale scenarios. For example, Fig. 7(a) contains 8 subgoals (vertices), resulting in 28 edges.

In contrast, MRSM-Hex adopts a hexagonal packing graph, where each vertex connects only to its neighboring nodes in a hexagonal grid pattern, reducing the number of edges to $\mathcal{O}(3|V|)$. Consequently, the computational complexity decreases to $\mathcal{O}(9|V|^2)$, significantly improving scalability. For instance, Fig. 7(b) contains 8 subgoals (vertices) but only 14 edges. However, this structured connectivity could restrict the available paths between nodes, and potentially limit the flexibility of MRSM-Hex in environments with complex obstacle layouts.

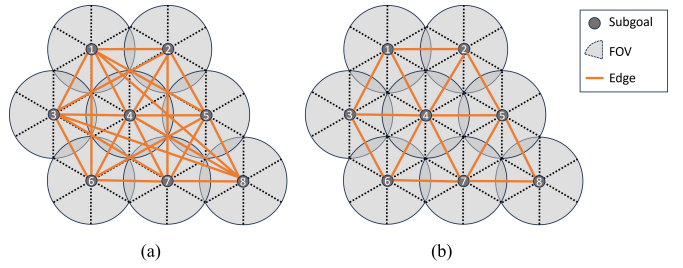


Figure 7. (a) A complete graph. (b) A hexagonal packing graph.

Algorithm 1 Multi-Robot Search with Matroid (MRSM)

Input: $G = (V, E, w)$ (weighted graph), $\mathbb{M}$ (set of matroidal independence systems), f (coverage function), $\mathcal{B}$ (balancing function), λ (balancing function weight), n (number of robots), l_i (routing budget)

Output: Selected edge set $S \subseteq E$

```

1:  $S \leftarrow \emptyset$ 
2:  $F \triangleq f + \lambda \mathcal{B}$ 
3:  $\mathbb{M} \triangleq \{\mathcal{M}_C, \mathcal{M}_R\}$ 
4: for  $i \leftarrow 1$  to  $n$  do
5:    $e_i \leftarrow \text{SampleMin}(E, S)$ 
6:    $S \leftarrow S \cup \{e_i\}$ 
7:    $E \leftarrow E \setminus \{e_i\}$ 
8: end for
9: while  $E \neq \emptyset$  do
10:   $e^* \leftarrow \arg \max_{e \in E} F(S \cup e) - F(S)$ 
11:   $S' \leftarrow S \cup \{e^*\}$ 
12:  if  $S' \in \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M}$  then
13:     $S \leftarrow S \cup \{e^*\}$ 
14:  end if
15:   $E \leftarrow E \setminus \{e^*\}$ 
16: end while

```

Experiments

To evaluate MRSM performance, there are three experiments: the target search (EX1), the parametric analysis

Algorithm 2 Dynamic MRSM (D-MRSM) under unknown maps

Input: $\mathbb{M}$ (set of matroidal independence systems), $\tilde{f}$ (approximated coverage function), $\mathcal{B}$ (balancing function), λ (balancing function weight), l_i (routing budget)

Output: Selected edge set S

```

1:  $S \leftarrow \emptyset$ 
2:  $F \triangleq \tilde{f} + \lambda B$ 
3:  $\mathbb{M} \triangleq \{\mathcal{M}_C, \mathcal{M}_R\}$ 
4: while each robot's trajectory length stays within  $l_i$  do
5:   Sample the edge set  $\tilde{E}$  from each robot's observation
6:   while  $\tilde{E} \neq \emptyset$  do
7:      $e^* \leftarrow \arg \max_{e \in \tilde{E}} F(S \cup e) - F(S)$ 
8:      $S' \leftarrow S \cup \{e^*\}$ 
9:     if  $S' \in \bigcap_{\mathcal{M} \in \mathbb{M}} \mathcal{M}$  then
10:       $S \leftarrow S'$ 
11:     end if
12:   end while
13: end while

```

(EX2), and the scalability analysis (EX3). The proposed algorithm (MRSM) and the benchmark algorithms (MRSIS (Li and Tseng 2024), CapAM (Paul et al. 2022), PD-FAC (Sheng et al. 2022), and the other 2 methods) are evaluated based on the expected number of detected objects (ENDO) and the ETDD. The ETDD is defined as follows:

$$E[TTD] = \frac{1}{TO} \sum_{t=1}^T \sum_{o=1}^O \tau_o^t,$$

where T , O , and τ_o^t represent the number of trials, number of objects in trial t , and detection time of object o in trial t , respectively. Besides, if the drones fail to detect objects, the search time is set to the maximum time constraint.

The search process is as follows: MRSM generates a set of trajectories composed of K_i edges before the i^{th} drone takes off. As the drone reaches the subgoal, it hovers and detects targets within 3 seconds. The drone continues to traverse the edge and visits the next subgoal. The process continues until all targets in the environment have been identified or all drones have traversed their designated edges. When the search task is completed, the drones land on the ground.

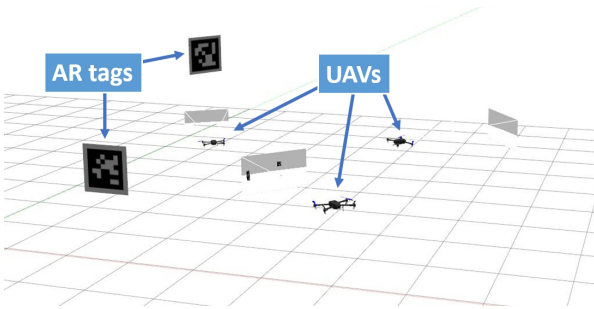


Figure 8. Gazebo simulations with 3 UAVs and 2 objects (AR tags) in a 3D Environment. The UAV can project a viewing frustum to observe the search space.

Experiment Setup

In the target search experiment (EX1), the performance of the proposed algorithms (MRSM and D-MRSM) is assessed through two simulation scenarios on Gazebo (see Fig. 8) and three realistic environments (see Fig. 9 and Fig. 10). The assessment includes a comparison with state-of-the-art approaches (e.g., MRSIS, CapAM, and PD-FAC). The simulation is carried out with three robots (UAVs) on Gazebo (see Fig. 8). All simulations run on an AMD Ryzen 9 7950X CPU and GeForce RTX 4080 GPU. Various environment sizes $\mathbb{E} = \{8, 12\}$ and numbers of targets (AR tags) $\mathbb{T} = \{2, 4, 6\}$ in the space are evaluated. Each (a, b) pair generates 100 trials with randomly located targets, where $a \in \mathbb{E}$ is an $a \times a \times a$ (m^3) search space, and $b \in \mathbb{T}$ is the number of targets in a . The robots' goal is to find all targets.

The first simulation scenario relies on a known map, while the second scenario considers an unknown environment. In the first simulation, a weighted graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, c)$ is constructed to represent the environment, where $\mathcal{V}$ is the subgoal set, $\mathcal{E}$ is the edge set, and c is the routing costs with respect to edges. The subgoals $\mathcal{V}$ are evenly distributed throughout the space and each subgoal is defined as $v \in \{(x, y, z, \theta) | (x, y, z, \theta) \in \mathcal{V}\}$, where $x, y, z \in [1, a]$ and $\theta \in \{0, 60, 120, 180, 240, 300\}$.

In the search process, each robot can take one of the actions from $\mathcal{A} = \{\text{Move}(x, y, z, \theta), \text{Detect}\}$. The *Move* action takes the drone to the coordinate (x, y, z) with the heading θ . The *Detect* action performs an object detection. The time constraint of the search process is 10 minutes. Once all the targets have been found or the search time exceeds the constraint, the task is terminated.

The coverage function[§] f is calculated by the ratio of the number of covered voxels to the total number of environmental voxels. The routing cost c is measured by the Euclidean distance between two subgoals[¶].

There are three real-world experiments with two drones. The first one is to search for three static targets in a $13 \times 9 \times 2$ m^3 public area on the third floor of the General Education Building at the National Central University (NCU) (see Fig. 9(a)). The space is divided into voxels whose unit size is $15 \times 15 \times 15$ cm^3 . Three targets (a sports ball, a chair, and a bottle) are randomly located and can be partially occluded due to the complex environment with obstacles (see Fig. 9(b)). The second and the third scenarios take place in a $50 \times 50 \times 5$ m^3 lawn at the Hengshan Calligraphy Art Park (see Fig. 10(a)). The difference is that the second scenario involves finding three static AR tags (see Fig. 10(b)), while the third scenario requires finding a moving AR tag traveling at 1 m/s in the environment. The space contains plenty of obstacles (trees). The edge set $\mathcal{E}$, consisting of obstacle-free edges, ensures that the robots do not crash into trees. (see Fig. 10(c))

The searchers are the drones developed by Taiwan Drone 100, shown in Fig. 11. The drone is equipped with NVIDIA Jetson Xavier NX, two Intel RealSense cameras, and a

[§]In the second scenario, the coverage at each subgoal is the same.

[¶]For any two connected nodes, $v_1 = (x_1, y_1, z_1, \theta_1)$ and $v_2 = (x_2, y_2, z_2, \theta_2)$, the routing cost $c(\{(v_1, v_2)\}) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2 + (\theta_1 - \theta_2)^2}$.

Here3+ GPS receiver. The Intel RealSense T265 camera, which uses visual simultaneous localization and mapping (vSLAM) to localize the drone, is mounted in the public area of the NCU, while the D435i camera is used to explore the environment. The camera and drone parameters are shown in Table 1. The Here3+ GPS, which supports real-time kinematic positioning (RTK), is used in the outdoor experiment at the Park.

To detect objects, the YOLOv5 (Jocher et al. 2022) is adopted and run on NVIDIA Jetson Xavier NX. Bayesian filtering is adopted to estimate the target detection in an environment. To successfully detect the object, the target must be within 3 m and the probability of detection must be above a threshold of 95% probability.

In the parametric analysis experiment (EX2), the parameters, the weight of the balancing function (λ) and the robot routing budget (l_i), are investigated for the non-tree-based structure (MRSIS) and the tree-based structure (MRSM) within an $8 \times 8 \times 8$ m³ simulated environment. To provide the balancing weight (λ) in various environment sizes $\mathbb{E} = \{8, 10, 12, 14\}$ are evaluated. Three robots (UAVs) are tasked with finding different numbers of targets $b \in \{2, 4, 6\}$ with different balancing weights $\lambda \in \{0, 0.2, 0.4, 0.6, 0.8, 1\}$ and robot routing budgets $l_i \in \{20, 40, 60, 80\}$. When $\lambda = 0$, no balanced workloads are considered. When $\lambda = 1$, the workloads assigned to robots are thoroughly optimized. The experiment setup is the same as the simulation in EX1. Additionally, a comparative analysis of computational time is conducted for both methods.

In the scalability analysis experiment (EX3), MRSM, MRSM-Hex, and benchmark approaches are evaluated under a $20 \times 20 \times 20$ m³ simulated space involving a number of robots $R = \{5, 10\}$. Performance is evaluated on different numbers of subgoals $\mathcal{V} = \{700, 800, 900\}$. Robots (UAVs) must find 8 targets (AR tags) in the environment. The balancing weight λ and robot routing budget l_i are set to 0.2 and 300, respectively. The camera range is set to 5 meters, and the remaining configurations are established in the same manner as the simulation in EX1.

Two baseline methods (CapAM and PD-FAC) are implemented for the search scenarios in both known and unknown environments. In the known environment scenario of CapAM, the state space contains features such as the elapsed mission time, the robot's current location, and the robot's work capacity. The action space of the MDP is the action set $\mathcal{A}$. In the unknown environment scenario, multiple neural networks are trained with varying numbers of task nodes. During the exploration phase, new tasks are introduced based on the robot's observations. The goal of CapAM is to visit as many subgoals as possible.

In the known environment scenario of PD-FAC, the weighted graph $\mathcal{G}$ maintains the same format as detailed in Sheng et al. (2022). In the unknown environment scenario, multiple neural networks are trained with different environmental layouts. During exploration, new nodes are introduced based on the robot's observations. The search time budget for both scenarios is set to 10 minutes, and the search targets are randomly placed on the graph nodes. Once the robot visits the node on the graph, the target is marked as

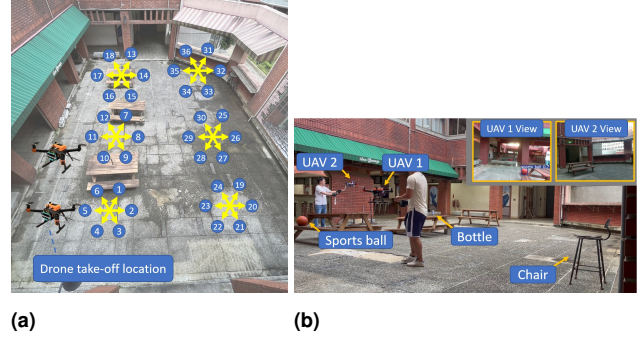


Figure 9. (a) The map of the $13 \times 9 \times 2$ m³ public area on the third floor of the General Education Building at the National Central University. (b) An example of UAVs searching for targets (a sports ball, a bottle, and a chair).

found. The configurations of PD-FAC can be found in Sheng et al. (2022).

Table 1. Parameters of EX1 and EX2.

Parameters	Simulation	Real world
Camera range	2 m	3 m
Horizontal FOV	45°	69°
Vertical FOV	45°	42°
Transitional velocity	1.3 m/sec	0.2 m/sec
Angular velocity	45 deg/sec	120 deg/sec

EX1: Comparison with Benchmarks on Targets Search

Table 2 and Table 3 present the simulated performance in known maps, while Table 4 and Table 5 show the results in unknown environments. The simulations are conducted with a routing budget $l_i = 350$, the number of robots $n = 3$, and a balancing function weight $\lambda = 1$ across various environment sizes. MRSM is compared to state-of-the-art baselines (e.g., MRSIS, CapAM, and PD-FAC).

In known environments, Table 2 shows that drones find more objects in a small environment ($a = 8$) compared to the larger one ($a = 12$). The decrease in the number of targets in the environment correlates with an increase in the difficulty of the search. MRSM achieves the highest ENDO in the large environment ($a = 12$). The advantage of MRSM lies in its ability to maximize both the coverage function and the workload balancing function under matroid constraints. In addition, the performance of MRSIS-TSP and MRSM is very close in the case of $a = 8$. The detailed performance of both MRSM and MRSIS-TSP will be examined in EX2. Table 3 shows the ETDD of all methods. All approaches are constrained to search within 10 minutes (600 seconds). For $a = 8$, MRSIS-MST is the most efficient, while for $a = 12$, MRSM takes the lead in efficiency.

In unknown environments, Table 4 shows that drones find fewer objects compared to known environments. Additionally, in Table 5, the absence of a map causes drones to take more time to explore the environment. However, D-MRSM is capable of locating targets in less average time.

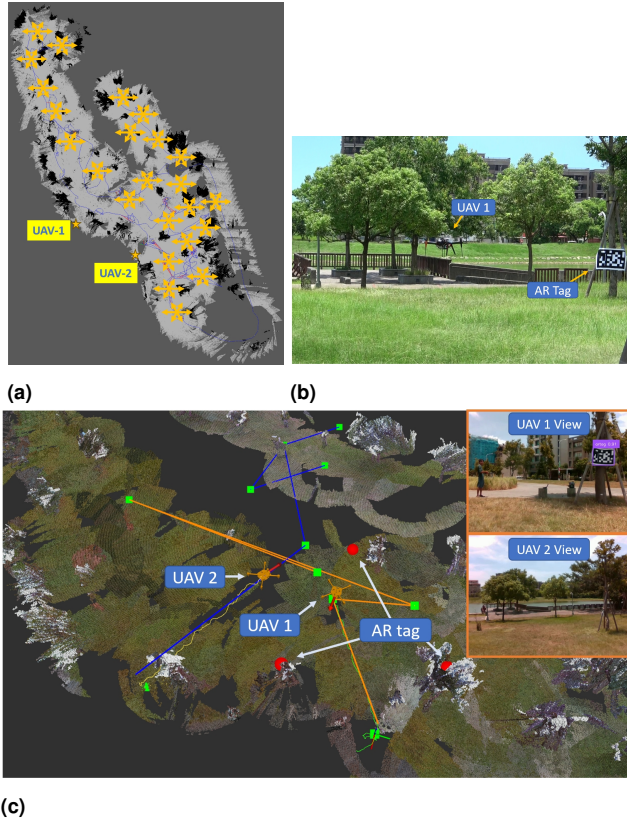


Figure 10. (a) The map of the $50 \times 50 \times 5 \text{ m}^3$ lawn at the Hengshan Calligraphy Art Park. The orange stars represent takeoff locations of two drones, while the orange arrows indicate the subgoal ground set. (b) An example of a drone searching for the AR tag. (c) Visualization of two drones performing the search task. The orange line and the blue line represent the trajectories of two drones. The green squares represent the subgoals. The red spheres are the locations of the AR tags.



Figure 11. A customized UAV developed by Taiwan Drone 100.

To further examine the proposed approaches, algorithms (MRSIS, CapAM, and PD-FAC) are evaluated in two real-world environments. Experiments are conducted 5 times for each approach.

In the National Central University scenario, the ETDD and the coverage rate are shown in Table 6. MRSM achieves an ETDD of 201 seconds and a coverage rate of 67%, which is the lowest ETDD and the highest coverage rate among other approaches. Unlike the MST approach (MRSIS-MST), MRSM generates spanning trees that may not be the shortest trajectories but are the most efficient for target discovery. The experiments demonstrate that MRSM outperforms the benchmark algorithms in ETDD.

In the Hengshan Calligraphy Art Park environment, the ETDD and the coverage rate are shown in Table 7 for 3 static AR tags and in Table 8 for a moving AR tag. In the static target scenario, MRSM outperforms the benchmarks in ETDD, achieving an ETDD of 196 seconds. Additionally, in the moving target scenario, MRSM surpasses the benchmarks, achieving an ETDD of 337 seconds. The results demonstrate that MRSM, traversing in a spanning tree structure, finds targets quickly in a large-scale environment.

The summaries of these experiments are as follows. The proposed algorithms (MRSM and D-MRSM) outperform state-of-the-art approaches (e.g., MRSIS, CapAM, and PD-FAC) as claimed in Thm. 10. MRSM and D-MRSM achieve the highest coverage rate and discover targets with the lowest ETDD. However, it is not guaranteed to find the most targets in the environment.

Table 2. The ENDO in a known map on the Gazebo simulator.

E	8			12		
	2	4	6	2	4	6
Random	0.86	1.43	2.25	0.12	0.34	0.52
CapAM	0.89	1.69	2.48	0.34	0.81	1.18
PD-FAC	0.49	0.81	1.11	0.15	0.43	0.46
MRSIS-MST	1.03	2.14	3.21	0.51	0.99	1.51
MRSIS-TSP	1.46	2.92	4.41	1.60	3.01	4.88
MRSM	1.44	2.92	4.39	1.68	3.19	4.97

Table 3. The ETDD in a known map on the Gazebo simulator. The unit is seconds.

E	8			12		
	2	4	6	2	4	6
Random	480	438	428	600	600	600
CapAM	463	418	377	544	500	463
PD-FAC	525	450	439	556	497	486
MRSIS-MST	251	211	205	532	478	454
MRSIS-TSP	264	273	259	379	416	386
MRSM	260	271	258	365	392	368

Table 4. The ENDO in an unknown map on the Gazebo simulator.

E	8			12		
	2	4	6	2	4	6
CapAM	0.53	1.44	2.36	0.27	0.96	1.15
PD-FAC	0.49	1.26	2.17	0.21	0.73	1.06
MRSIS-MST	0.73	1.65	2.53	0.41	1.10	1.57
MRSIS-TSP	0.62	1.63	2.75	0.47	1.23	1.71
D-MRSM	0.98	1.96	3.09	0.47	1.30	1.76

EX2: Parametric Analysis

To verify the advantage of the tree structure and the significance of the balancing function, MRSIS-TSP and MRSM are compared under various balancing weights (λ)

Table 5. The ETDD in an unknown map on the Gazebo simulator. The unit is seconds.

$\mathbb{E}$	8			12		
	2	4	6	2	4	6
CapAM	583	564	553	593	592	596
PD-FAC	592	559	581	597	595	597
MRSIS-MST	566	534	531	591	595	587
MRSIS-TSP	515	519	527	585	593	593
D-MRSM	503	506	520	582	585	581

Table 6. The ETDD of MRSM and baselines at the National Central University.

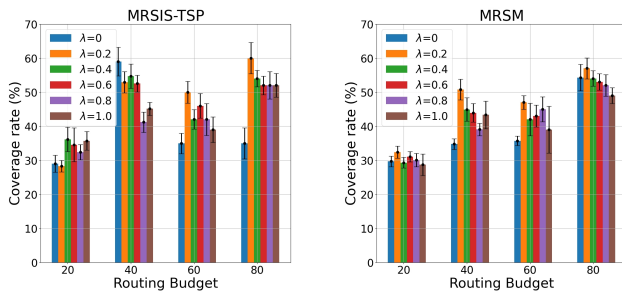
Method	Mean (sec.)	Std.	Coverage Rate
CapAM	275	93	42%
PD-FAC	314	130	33%
MRSIS-MST	206	75	65%
MRSIS-TSP	280	58	62%
MRSM	201	93	67%

Table 7. The ETDD of MRSM and baselines at the Hengshan Calligraphy Art Park with 3 AR tags.

Method	Mean (sec.)	Std.	Coverage Rate
CapAM	299	63	22%
PD-FAC	268	55	27%
MRSIS-MST	230	43	29%
MRSIS-TSP	385	58	25%
MRSM	196	31	31%

Table 8. The ETDD of MRSM and baselines at the Hengshan Calligraphy Art Park with a moving target.

Method	Mean (sec.)	Std.	Coverage Rate
CapAM	397	103	22%
PD-FAC	382	93	27%
MRSIS-MST	363	65	29%
MRSIS-TSP	370	96	25%
MRSM	337	83	31%



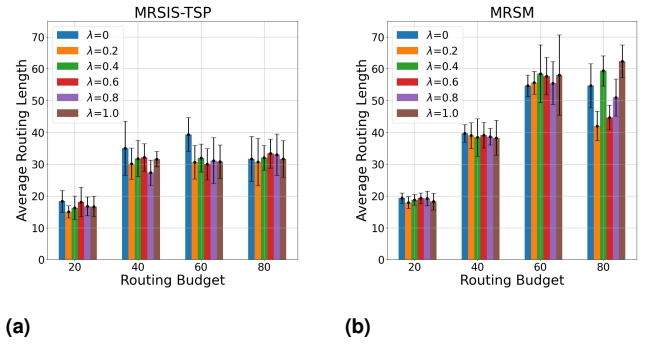
(a)

(b)

Figure 12. Coverage rate with different robot routing budget l_i and balancing weight λ .

and robot routing budgets (l_i). The coverage rate and the routing length are as follows.

Fig. 12(a) illustrates the coverage rate of MRSIS-TSP. When $\lambda = 0$, a substantial variance exists among the routing budgets. The absence of a balancing function



(a)

(b)

Figure 13. Average routing length (in meters) with various balancing weights λ and robot routing budget.**Table 9.** The ENDO with different parameters (robot routing budget l_i , number of targets $\mathbb{T}$, and balancing weight λ) on Gazebo simulator.

l_i	$\mathbb{T}$	Method	λ					
			0	0.2	0.4	0.6	0.8	1
20	2	MRSIS-TSP	0.8	1.0	1.1	1.0	1.1	1.1
		MRSM	1.1	1.1	1.2	1.2	1.1	1.1
	4	MRSIS-TSP	1.8	2.1	2.3	2.3	2.2	2.0
		MRSM	2.3	2.2	2.3	2.4	2.2	2.2
	6	MRSIS-TSP	2.6	3.2	3.4	3.4	3.2	3.1
		MRSM	3.4	3.3	3.4	3.6	3.2	3.3
40	2	MRSIS-TSP	1.5	1.3	1.4	1.4	1.2	1.5
		MRSM	1.2	1.4	1.4	1.4	1.3	1.3
	4	MRSIS-TSP	3.2	2.8	2.8	3.1	2.6	3.0
		MRSM	2.4	2.9	2.8	2.9	2.7	2.8
	6	MRSIS-TSP	4.6	4.1	4.2	4.5	3.8	4.5
		MRSM	3.5	4.5	4.3	4.3	4.1	4.1
60	2	MRSIS-TSP	1.3	1.0	1.0	1.0	1.0	1.0
		MRSM	1.3	0.9	0.9	0.9	0.9	0.9
	4	MRSIS-TSP	2.6	2.2	2.0	2.2	2.2	2.1
		MRSM	2.7	2.1	1.9	2.0	2.0	1.8
	6	MRSIS-TSP	3.7	3.3	3.0	3.2	3.3	3.2
		MRSM	4.0	3.1	2.9	2.9	3	2.8
80	2	MRSIS-TSP	1.3	1.5	1.5	1.4	1.4	1.4
		MRSM	1.5	1.4	1.5	1.5	1.4	1.4
	4	MRSIS-TSP	2.7	3.2	3.0	2.8	2.9	3.1
		MRSM	3.1	3.1	3.0	3.1	3.0	2.9
	6	MRSIS-TSP	4	4.6	4.4	4.1	4.3	4.4
		MRSM	4.7	4.4	4.4	4.5	4.4	4.3

($\lambda = 0$) outperforms at $l_i = 40$ while performing less effectively in other cases. Besides, a smaller balancing weight demonstrates effective performance. At $l_i = 20$, $l_i = 60$, and $l_i = 80$, a small magnitude of the balancing weight performs optimally. At $l_i = 40$, the absence of a balancing function ($\lambda = 0$) ranks the highest. However, when a balancing function is considered, $\lambda = 0.4$ achieves the highest coverage rate. Fig. 12(b) illustrates the coverage rate of MRSM. The weight $\lambda = 0.2$ yields the highest coverage rate across all routing budget cases.

Table 10. The ETDD with different parameters (robot routing budget l_i , number of targets $\mathbb{T}$, and balancing weight λ) on Gazebo simulator. The unit is in seconds.

l_i	$\mathbb{T}$	Method	λ					
			0	0.2	0.4	0.6	0.8	1
20	2	MRSIS-TSP	384	342	335	347	328	342
		MRSM	326	315	303	282	323	328
	4	MRSIS-TSP	382	340	313	325	337	355
		MRSM	308	313	315	292	317	328
	6	MRSIS-TSP	378	334	313	316	330	348
		MRSM	309	313	314	292	327	320
40	2	MRSIS-TSP	272	302	282	271	316	258
		MRSM	309	261	278	261	293	300
	4	MRSIS-TSP	251	275	289	249	292	261
		MRSM	329	264	286	252	279	273
	6	MRSIS-TSP	260	278	277	249	292	247
		MRSM	320	246	255	244	277	269
60	2	MRSIS-TSP	307	266	270	283	266	297
		MRSM	308	317	318	307	324	328
	4	MRSIS-TSP	310	195	203	223	205	226
		MRSM	312	229	271	243	238	249
	6	MRSIS-TSP	321	159	158	190	148	179
		MRSM	301	205	231	194	209	204
80	2	MRSIS-TSP	313	277	273	286	275	263
		MRSM	269	288	274	273	286	280
	4	MRSIS-TSP	293	256	275	288	266	242
		MRSM	286	257	280	267	279	290
	6	MRSIS-TSP	291	260	274	286	256	249
		MRSM	271	270	271	262	269	278

Table 11. Computational time of MRSIS-TSP and MRSM under different routing budgets. The unit is in seconds.

l_i	Method	λ					
		0	0.2	0.4	0.6	0.8	1
20	MRSIS-TSP	243	283	281	294	318	359
	MRSM	149	172	176	174	175	163
40	MRSIS-TSP	337	396	395	446	424	396
	MRSM	146	157	198	191	198	193
60	MRSIS-TSP	326	455	473	441	459	444
	MRSM	103	155	184	185	179	185
80	MRSIS-TSP	323	461	506	481	477	445
	MRSM	88	156	176	178	180	180

Table 12. The ENDO of MRSM with different parameters (environment sizes $\mathbb{E}$, number of targets $\mathbb{T}$, and balancing weights λ) on Gazebo simulator.

$\mathbb{E}$	$\mathbb{T}$	λ					
		0	0.2	0.4	0.6	0.8	1
2	2	1.62	1.96	1.96	1.96	1.96	1.91
	4	3.18	3.87	3.88	3.90	3.86	3.92
	6	4.91	5.90	5.91	5.89	5.85	5.87
10	2	1.55	1.68	1.53	1.60	1.54	1.57
	4	2.80	3.19	3.16	3.23	3.06	3.33
	6	4.25	4.88	4.72	4.98	4.58	5.04
12	2	1.02	0.92	1.09	0.97	1.08	0.97
	4	1.83	1.99	2.07	1.89	2.25	1.74
	6	2.88	3.03	3.13	2.84	2.90	2.75
14	2	0.68	0.62	0.58	0.61	0.64	0.54
	4	1.18	1.04	1.00	1.19	1.01	1.02
	6	1.65	1.77	1.62	1.69	1.68	1.46

Fig. 13 illustrates the average routing length on three robots with various balancing weights (λ) and robot routing**Table 13.** The ETDD of MRSM with different parameters (environment sizes $\mathbb{E}$, number of targets $\mathbb{T}$, and balancing weights λ) on Gazebo simulator. The unit is in seconds.

$\mathbb{E}$	$\mathbb{T}$	λ					
		0	0.2	0.4	0.6	0.8	1
8	2	200	125	127	128	122	141
	4	207	130	135	125	133	131
	6	196	121	129	128	129	131
10	2	217	179	212	190	208	209
	4	257	202	198	184	211	183
	6	251	195	199	173	212	180
12	2	337	360	313	345	320	344
	4	366	340	332	353	310	370
	6	354	337	329	351	352	357
14	2	420	435	447	440	431	458
	4	445	461	468	445	468	465
	6	455	443	457	451	453	471

budgets (l_i). As the routing budget increases, robots cover larger traveling distances. The routing length converges due to the limitation of the environment size. In the MRSIS-TSP scenario (see Fig. 13(a)), robots travel between 10 and 40 meters. In the MRSM scenario (see Fig. 13(b)), the traveled distance is between 10 and 65 meters. The reason for the smaller routing distance in MRSIS-TSP compared to MRSM is that MRSIS-TSP utilizes a TSP approximator to minimize travel distances. Therefore, the average routing length of MRSM is more than that of MRSIS-TSP.

Table 9 and Table 10 show the ENDO and the ETDD, respectively. The best performance is highlighted in bold. The results indicate that no single method consistently outperforms in all cases. On average, MRSM outperforms in 38 out of 72 cases, while MRSIS-TSP achieves superiority in 34 out of 72 cases. Additionally, MRSM excels in small routing budgets ($l_i \leq 40$), while MRSIS-TSP performs well in the larger ones. Since MRSIS-TSP inherits the routing minimization feature, it has an advantage in larger routing budgets by facilitating efficient travel. Moreover, in some cases, the performance is superior when the balancing function is not involved. Since the proposed objective function is defined as $F(S) = f(S) + \lambda B(S)$, when $\lambda = 0$, the objective function is simply a coverage function. Optimizing the coverage function with routing and clustering constraints may yield good performance in certain cases.

Table 11 illustrates the computational time with respect to routing budgets. MRSM achieves significantly shorter computational times than MRSIS-TSP. In addition, as the routing budget increases, the computational time of MRSIS-TSP increases. The reason is that MRSIS-TSP plans trajectories with a TSP approximator at each iteration, while MRSM directly selects the next best marginal gain. As a result, MRSM computes much faster, even in large search spaces.

To elucidate the use of the balancing weight (λ), Table 12 and Table 13 show the performance (the ENDO and the ETDD) of MRSM in various environment sizes and number of targets. Small environments allow for faster target detection compared to large ones. Although no single λ value is optimal for all environment sizes, empirical results indicate that small environments benefit from larger λ values,

whereas large environments can perform well with smaller λ values. As the environment size increases, the results suggest that a large area can be covered even with an imbalance in workload distribution.

In EX2, smaller balancing weights generally improve performance for MRSM and MRSIS-TSP, although in some settings larger weights yield the best results. In most cases, MRSM outperforms MRSIS-TSP due to the theoretical guarantee of submodularity under matroid constraints. In contrast, small environments require a more balanced workload assignment to optimize performance.

EX3: Scalability Analysis

Table 14 shows the performance of MRSM, MRSM-Hex, MRSIS, CapAM, and PD-FAC with various numbers of robots (UAVs) and subgoals. The coverage rate increases with the number of subgoals. The results show that MRSM outperforms the benchmarks in ENDO, ETDD, and coverage rate. Note that the performance of MRSM and MRSIS is close. However, MRSM exceeds MRSIS due to its better performance guarantee. Furthermore, MRSM-Hex's performance is close to MRSM's performance with a dramatically lower computational time.

Fig. 14 illustrates the computational time of MRSM, MRSM-Hex, and benchmark algorithms with 5 UAVs and 10 UAVs. Since CapAM and PD-FAC are neural network-based and trained on GPUs, their computational time is faster compared to MRSM and MRSIS, which are CPU-bound. However, MRSM-Hex outperforms the others in speed due to a significant reduction in the edge set. Additionally, the computational time of MRSM is faster than that of MRSIS for both 5 UAVs and 10 UAVs, as MRSM eliminates redundant processes (solving TSP or MST problems). It is worth noting that the computational time of MRSIS increases significantly as the number of subgoals grows.

In summary, for MRSM approach, as the number of subgoals and robots increases, the overall performance does not deteriorate due to the theoretical guarantee of submodularity. In addition, MRSM outperforms state-of-the-art methods for multi-robot search scenarios.

Discussion on Scalability

The similar performance of MRSM and MRSM-Hex in EX3 suggests that the reduced edge set in MRSM-Hex does not compromise solution quality while providing lower computation. Since both methods tend to greedily select the next subgoal from the current one, the use of a complete graph is redundant. For instance, MRSM uses 244,650 edges for 700 subgoals, 319,600 edges for 800 subgoals, and 404,550 edges for 900 subgoals, respectively. In contrast, MRSM-Hex only requires 2,036, 2,300, and 2,653 edges for 700, 800, and 900 subgoals, respectively.

Despite the differences in graph structure, the theoretical guarantee established in Thm. 10 remains valid, which is relevant to the ground set. The major difference between MRSM and MRSM-Hex is the distinct ground sets. The ground set of MRSM is the edges of the complete graph, while the ground set of MRSM-Hex is the edges of the hexagonal packing graph (see Fig. 7.) For example, assume the optimal solutions for the complete graph and the

Table 14. Large-scale experiment results of the ENDO, the ETDD, and the coverage rate with different numbers of subgoals and UAVs on the Gazebo simulator.

No. Subgoals	No. UAVs	Method	ENDO	ETDD	Coverage rate
700	5	CapAM	5.49	551	66%
		PD-FAC	3.87	413	47%
		MRSIS-MST	5.43	310	67%
		MRSIS-TSP	5.72	320	68%
		MRSM	6.24	290	73%
		MRSM-Hex	5.83	307	71%
	10	CapAM	6.13	514	73%
		PD-FAC	4.52	381	53%
		MRSIS-MST	6.13	250	73%
		MRSIS-TSP	5.83	266	72%
		MRSM	6.23	238	75%
		MRSM-Hex	6.19	245	73%
800	5	CapAM	4.68	514	58%
		PD-FAC	4.75	383	56%
		MRSIS-MST	5.86	293	67%
		MRSIS-TSP	6.12	302	72%
		MRSM	6.42	280	75%
		MRSM-Hex	6.20	290	73%
	10	CapAM	6.17	534	73%
		PD-FAC	4.35	376	59%
		MRSIS-MST	6.14	307	72%
		MRSIS-TSP	6.32	293	75%
		MRSM	6.43	257	77%
		MRSM-Hex	6.36	265	76%
900	5	CapAM	4.92	496	58%
		PD-FAC	4.08	416	51%
		MRSIS-MST	6.76	295	73%
		MRSIS-TSP	6.3	284	71%
		MRSM	6.87	256	77%
		MRSM-Hex	6.75	275	74%
	10	CapAM	5.98	506	72%
		PD-FAC	4.4	354	53%
		MRSIS-MST	6.74	255	78%
		MRSIS-TSP	6.17	293	73%
		MRSM	6.93	243	82%
		MRSM-Hex	6.83	253	80%

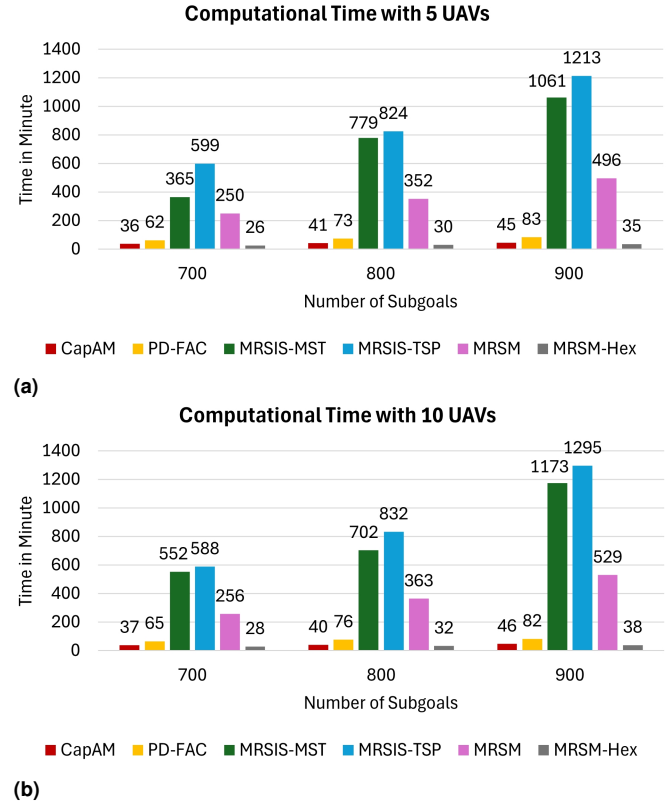


Figure 14. Large-scale experiment results of the computational time on the Gazebo simulator.

hexagonal packing graph are $\widetilde{S}_{CG}^{opt}$ and $\widetilde{S}_{Hex}^{opt}$, respectively. The theoretical guarantees of MRSM and MRSM-Hex are $\frac{1}{3}F(\widetilde{S}_{CG}^{opt})$ and $\frac{1}{3}F(\widetilde{S}_{Hex}^{opt})$, respectively.

Conclusions

To overcome multi-robot search problems consisting of maximal coverage, task allocation, and minimal routing problems, this research proposes an MRSM algorithm. The maximal coverage and task allocation problems are solved by reformulating the clustering and routing constraints into the intersection of matroids. Rather than solving the routing problem with TSP or MST approximator, MRSM expands a spanning tree by selecting the next node with the maximal marginal gain, leading to a faster computational time. With submodularity and matroid properties, MRSM achieves $\frac{1}{3}\widetilde{OPT}$, where $\widetilde{OPT}$ is the optimal performance under spanning-tree structures (e.g., complete graph or hexagonal packing graph). Simulated and real-world experiment results show that MRSM approaches outperform state-of-the-art approaches regarding ETTD in multi-robot search problems and scale effectively as the search space grows. Furthermore, the performance of various parameters is provided to facilitate deployment in specific applications.

The future work of this research is as follows. First, although D-MRSM outperforms the benchmarks, its theoretical performance remains unknown. Moreover, the coverage function in the objective is approximated, which could be enhanced by incorporating the compressed sensing method (Lu and Tseng 2020). Besides, extending MRSM to plan a trajectory online via adaptive submodularity is another direction (Golovin and Krause 2011). Second, in EX2, where MRSM does not consistently surpass MRSIS-TSP, exploring the integration of both advantages becomes another viable direction. Third, in EX3, although MRSM-Hex reduces computational complexity, its reduced edge set limits flexibility. Developing a parallel or online algorithm for MRSM is a potential direction for improvement (Breuer et al. 2020).

Acknowledgements

This research was completed thanks to the financial support from Taiwan NSTC Grant 112-2221-E-008-075.

References

- Aironi C, Cornell S and Squartini S (2022) Tackling the linear sum assignment problem with graph neural networks. *International Conference on Applied Intelligence and Informatics* : 90–101.
- Breuer A, Balkanski E and Singer Y (2020) The fast algorithm for submodular maximization. *International Conference on Machine Learning* : 1134–1143.
- Corah M and Michael N (2019) Distributed matroid-constrained submodular maximization for multi-robot exploration: Theory and practice. *Autonomous Robots* 43: 485–501.
- Correll N and Martinoli A (2009) Multirobot inspection of industrial machinery. *IEEE Robotics & Automation Magazine* 16(1): 103–112.
- Fisher ML, Nemhauser GL and Wolsey LA (1978) An analysis of approximations for maximizing submodular set functions—ii : 73–87.
- Goarin M and Loiano G (2024) Graph neural network for decentralized multi-robot goal assignment. *IEEE Robotics and Automation Letters* 9(5): 4051–4058.
- Golovin D and Krause A (2011) Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research* 42: 427–486.
- Gunawan A, Lau HC and Vansteenwegen P (2016) Orienteering problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research* 255(2): 315–332.
- Jennings JS, Whelan G and Evans WF (1997) Cooperative search and rescue with a team of mobile robots. *IEEE International Conference on Advanced Robotics (ICAR)* : 193–200.
- Jocher G, Chaurasia A, Stoken A, Borovec J, NanoCode012, Kwon Y, Michael K, TaoXie, Fang J, imyhxy, Lorna, Yifu Z, Wong C, V A, Montes D, Wang Z, Fati C, Nadar J, Laughing, UnglvKitDe, Sonck V, tkianai, yxNONG, Skalski P, Hogan A, Nair D, Strobel M and Jain M (2022) ultralytics/yolov5: v7.0 - YOLOv5 SOTA Realtime Instance Segmentation. DOI: 10.5281/zenodo.7347926. URL <https://doi.org/10.5281/zenodo.7347926>.
- Jorgensen S, Chen RH, Milam MB and Pavone M (2017a) The matroid team surviving orienteers problem: Constrained routing of heterogeneous teams with risky traversal. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* : 5622–5629.
- Jorgensen S, Chen RH, Milam MB and Pavone M (2017b) The team surviving orienteers problem: Routing robots in uncertain environments with survival constraints. *First IEEE International Conference on Robotic Computing (IRC)* : 227–234.
- Kaempfer Y and Wolf L (2018) Learning the multiple traveling salesmen problem with permutation invariant pooling networks. *arXiv preprint arXiv:1803.09621* .
- Khuller S, Moss A and Naor JS (1999) The budgeted maximum coverage problem. *Information processing letters* 70(1): 39–45.
- Kool W, van Hoof H and Welling M (2019) Attention, learn to solve routing problems! *International Conference on Learning Representations* .
- Korsah GA, Stentz A and Dias MB (2013) A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research* 32(12): 1495–1512.
- Korte B and Hausmann D (1978) An analysis of the greedy heuristic for independence systems. *Annals of Discrete Mathematics* 2: 65–74.
- Krause A and Guestrin C (2007) Near-optimal observation selection using submodular functions. *Association for the Advancement of Artificial Intelligence (AAAI)* 7: 1650–1654.
- Kruskal JB (1956) On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical society* 7(1): 48–50.
- Lauri M, Pajarinen J, Peters J and Frintrap S (2020) Multi-sensor next-best-view planning as matroid-constrained submodular maximization. *IEEE Robotics and Automation Letters* 5(4): 5323–5330.

- Lee M, Xiong Y, Yu G and Li GY (2018) Deep neural networks for linear sum assignment problems. *IEEE Wireless Communications Letters* 7(6): 962–965.
- Li YS and Tseng KS (2024) Multi-robot search in a 3d environment with intersection system constraints. *IEEE International Conference on Robotics and Automation (ICRA)* : 5963–5969.
- Lin PT and Tseng KS (2023) Improvement of submodular maximization problems with routing constraints via submodularity and fourier sparsity. *IEEE Robotics and Automation Letters* 8(4): 1927–1934.
- Liu J and Williams RK (2018) Optimal intermittent deployment and sensor selection for environmental sensing with multi-robot teams. *IEEE International Conference on Robotics and Automation (ICRA)* : 1078–1083.
- Liu J and Williams RK (2019) Submodular optimization for coupled task allocation and intermittent deployment problems. *IEEE Robotics and Automation Letters* 4(4): 3169–3176.
- Liu MY, Tuzel O, Ramalingam S and Chellappa R (2013) Entropy-rate clustering: Cluster analysis via maximizing a submodular function subject to a matroid constraint. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36(1): 99–112.
- Lu BX and Tseng KS (2020) 3d map exploration via learning submodular functions in the fourier domain. *IEEE International Conference on Unmanned Aircraft Systems (ICUAS)* : 1199–1205.
- Martello S and Toth P (1987) Linear assignment problems. *North-Holland Mathematics Studies* 132: 259–282.
- Mestre J (2015) On the intersection of independence systems. *Operations Research Letters* 43(1): 7–9.
- Mittal M, Mohan R, Burgard W and Valada A (2019) Vision-based autonomous uav navigation and landing for urban search and rescue. *The International Symposium of Robotics Research* : 575–592.
- Mohamed SC, Fung A and Nejat G (2022) A multirobot person search system for finding multiple dynamic users in human-centered environments. *IEEE Transactions on Cybernetics* : 1–13.
- Mohamed SC, Rajaratnam S, Hong ST and Nejat G (2020) Person finding: An autonomous robot search method for finding multiple dynamic users in human-centered environments. *IEEE Transactions on Automation Science and Engineering* 17(1): 433–449.
- Nemhauser GL, Wolsey LA and Fisher ML (1978) An analysis of approximations for maximizing submodular set functions—i. *Mathematical programming* 14(1): 265–294.
- Paul S, Ghassemi P and Chowdhury S (2022) Learning scalable policies over graphs for multi-robot task allocation using capsule attention networks. *IEEE International Conference on Robotics and Automation (ICRA)* : 8815–8822.
- Qi CR, Su H, Mo K and Guibas LJ (2017) Pointnet: Deep learning on point sets for 3d classification and segmentation. *IEEE conference on computer vision and pattern recognition* : 652–660.
- Ralphs TK, Kopman L, Pulleyblank WR and Trotter LE (2003) On the capacitated vehicle routing problem. *Mathematical programming* 94: 343–359.
- Schlotfeldt B, Thakur D, Atanasov N, Kumar V and Pappas GJ (2018) Anytime planning for decentralized multirobot active information gathering. *IEEE Robotics and Automation Letters* 3(2): 1025–1032.
- Sheng W, Guo H, Yau WY and Zhou Y (2022) Pd-fac: Probability density factorized multi-agent distributional reinforcement learning for multi-robot reliable search. *IEEE Robotics and Automation Letters* 7(4): 8869–8876.
- Singh A, Krause A, Guestrin C, Kaiser W and Batalin M (2007) Efficient planning of informative paths for multiple robots. *International joint conference on Artificial intelligence (IJCAI)* : 2204–2211.
- Tolstaya E, Paulos J, Kumar V and Ribeiro A (2021) Multi-robot coverage and exploration using spatial graph neural networks. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* : 8944–8950.
- Tsai YC, Lu BX and Tseng KS (2019) Spatial search via adaptive submodularity and deep learning. *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)* : 112–113.
- Williams RK, Gasparri A and Ulivi G (2017) Decentralized matroid optimization for topology constraints in multi-robot allocation problems. *IEEE International Conference on Robotics and Automation (ICRA)* : 293–300.
- Zhang H, Cheng J, Zhang L, Li Y and Zhang W (2022a) H2gcn: hierarchical-hops graph neural networks for multi-robot exploration in unknown environments. *IEEE Robotics and Automation Letters* 7(2): 3435–3442.
- Zhang H, Li R, Wu Z and Sun G (2022b) Nonmonotone submodular maximization under routing constraints. *arXiv preprint arXiv:2211.17131* .
- Zhang H and Vorobeychik Y (2016) Submodular optimization with routing constraints. *Association for the Advancement of Artificial Intelligence (AAAI)* 30(1).
- Zheng K, Sung Y, Konidaris G and Tellex S (2021) Multi-resolution pomdp planning for multi-object search in 3d. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* : 2022–2029.
- Zhou B, Xu H and Shen S (2023) Racer: Rapid collaborative exploration with a decentralized multi-uav system. *IEEE Transactions on Robotics* 39(3): 1816–1835.
- Zhou B, Zhang Y, Chen X and Shen S (2021) Fuel: Fast uav exploration using incremental frontier structure and hierarchical planning. *IEEE Robotics and Automation Letters* 6(2): 779–786.
- Zhu X, Vanegas F and Gonzalez F (2020) An approach for multi-uav system navigation and target finding in cluttered environments. *International Conference on Unmanned Aircraft Systems (ICUAS)* : 1113–1120.